

Cloud Design Patterns - Presentation Script

Slide 1 & 2: Title & Content

Action: Present yourself as a CloudOps Engineer and introduce the agenda.

Español (Guion)	English (Script)
Hola a todos, mi nombre es Juan David Pacheco Vargas. Soy Ingeniero CloudOps y el día de hoy les voy a hablar sobre los Patrones de Diseño Cloud. En esta presentación, cubriremos qué son estos patrones, exploraremos el patrón de Cola de Prioridad (Priority Queue) en sus variantes de cola única y múltiple, y finalmente analizaremos el patrón de Throttling, incluyendo recomendaciones y cuándo es ideal implementarlos.	Hello everyone, my name is Juan David Pacheco Vargas. I'm a CloudOps Engineer, and today I will be talking to you about Cloud Design Patterns. In this presentation, we will cover what these patterns are, explore the Priority Queue pattern in both single and multiple queue variants, and finally dive into the Throttling pattern, including best practices and when to use them.

Slide 3: What are they?

Español (Guion)	English (Script)
Para empezar, ¿qué son los patrones de diseño en la nube? Son soluciones probadas e independientes de la tecnología para superar los desafíos arquitectónicos comunes en cualquier entorno de nube. Nos ayudan a garantizar disponibilidad, resiliencia, escalabilidad y seguridad de manera eficiente.	To start, what are cloud design patterns? They are proven, technology-agnostic solutions used to overcome common architectural challenges in any cloud environment. They help us efficiently ensure availability, resiliency, scalability, and security.

Slide 4 & 5: Priority Queue Pattern & Single Queue

Español (Guion)

El patrón de Cola de Prioridad utiliza colas de mensajes para procesar tareas urgentes más rápido que las de menor prioridad, garantizando así que se cumplan los SLAs críticos. En el enfoque de "Single Queue" o Cola Única, la aplicación asigna una prioridad a cada mensaje y los envía todos a una misma cola, donde son ordenados internamente.

English (Script)

The Priority Queue pattern uses message queues to process urgent tasks faster than lower-priority ones, ensuring critical service-level agreements are met. In a "Single Queue" approach, the application assigns a priority to each message and sends them all to one single queue, where they are ordered internally.

Slide 6: Single Queue Example

Action: Point to the terminal output on the slide and explain the behavior.

Español (Guion)

Como podemos observar en la salida del código de ejemplo, enviamos tres mensajes con diferentes prioridades. Aunque los ingresamos en el orden: tarea baja, tarea urgente, y tarea normal; al momento de procesarlos, el sistema atiende primero la 'Tarea URGENTE' (Prioridad 1), seguida de la normal y por último la baja.

English (Script)

As we can see in the example code output, we send three messages with different priorities. Even though they are sent in the order: low, urgent, and normal task; when processing begins, the system handles the 'URGENT task' (Priority 1) first, followed by the normal task, and finally the low priority one.

```
[SEND] Mensaje 'Tarea baja' con prioridad 3 añadido.  
[SEND] Mensaje 'Tarea URGENTE' con prioridad 1 añadido.  
[SEND] Mensaje 'Tarea normal' con prioridad 2 añadido.
```

```
--- Procesando mensajes (Single Queue) ---  
[PROCESS] Procesando: 'Tarea URGENTE' (Prioridad 1)  
[PROCESS] Procesando: 'Tarea normal' (Prioridad 2)  
[PROCESS] Procesando: 'Tarea baja' (Prioridad 3)
```

Slide 7 & 8: Multiple Queues & Example

Español (Guion)

Por otro lado, tenemos el enfoque de Colas Múltiples. Aquí, cada mensaje tiene su valor de prioridad, pero se envían a colas separadas según esa prioridad. Pueden compartir un solo grupo de consumidores o tener consumidores dedicados. En nuestra demostración, enviamos múltiples tareas a sus colas respectivas. Como ven en el resultado, el consumidor procesa primero todas las tareas de la cola de prioridad 1 (las dos urgentes) antes de pasar a la prioridad 2 y 3.

English (Script)

On the other hand, we have the Multiple Queues approach. Here, each message is assigned a priority value, but different priorities use separate queues. They can share a single consumer pool or have dedicated ones. In our demo, we send multiple tasks to their respective queues. As you can see in the output, the consumer processes all tasks from the priority 1 queue (both urgent ones) before moving to priorities 2 and 3.

```
[SEND] Mensaje 'Tarea baja' añadido a cola de prioridad 3.  
[SEND] Mensaje 'Tarea URGENTE' añadido a cola de prioridad 1.  
[SEND] Mensaje 'Tarea normal' añadido a cola de prioridad 2.  
[SEND] Mensaje 'Otra URGENTE' añadido a cola de prioridad 1.
```

```
--- Procesando mensajes (Multiple Queue) ---  
[PROCESS] Procesando: 'Tarea URGENTE' (Prioridad 1)  
[PROCESS] Procesando: 'Otra URGENTE' (Prioridad 1)  
[PROCESS] Procesando: 'Tarea normal' (Prioridad 2)  
[PROCESS] Procesando: 'Tarea baja' (Prioridad 3)
```

Slide 9 & 10: When to Use/Avoid & Recommendations

Español (Guion)

Deberíamos usar este patrón cuando distintos flujos de trabajo requieren diferentes velocidades, por ejemplo, usuarios premium frente a estándar, o durante picos de tráfico. Sin embargo, evítenlo si todas las tareas son igual de importantes y basta con un modelo FIFO, o si las tareas dependen unas de otras. Como recomendación: establezcan prioridades claras, monitoreen la latencia para evitar bloqueos y controlen los costos escalando los recursos según la demanda.

English (Script)

We should use this pattern when different workflows require different speeds, like premium vs. standard users, or during traffic spikes. However, avoid it if all tasks are equally important and FIFO works fine, or if tasks depend on each other. As best practices: set clear priorities, monitor latency to avoid bottlenecks, and control costs by scaling resources based on demand.

Slide 11, 12 & 13: Throttling & Considerations

Español (Guion)

Pasando al patrón de Throttling, este nos permite controlar y limitar el consumo de recursos de una aplicación, un cliente (tenant) o un servicio completo. Es crucial diseñarlo identificando el punto de saturación y reaccionar rápidamente rechazando peticiones excesivas (por ejemplo, con un código 429). Se utiliza principalmente para mantener los SLOs del sistema, evitar que un solo usuario monopolice los recursos y manejar picos de actividad repentinos.

English (Script)

Moving on to the Throttling pattern, this allows us to control and limit the resource consumption of an application, an individual tenant, or an entire service. It's crucial to architect it by finding the saturation point and reacting early by rejecting excess requests (like returning a 429 status). It's primarily used to keep systems within their SLOs, prevent a single tenant from monopolizing resources, and handle sudden bursts of activity.

Slide 15: Throttling Example

Action: Explain the demonstration comparing the Premium and Free tenants.

Español (Guion)	English (Script)
En nuestra demostración de Throttling, configuramos dos clientes: Premium con 5 peticiones por segundo y Free con 2. Al ejecutar la simulación, podemos notar que ambos clientes son aceptados inicialmente. Sin embargo, cuando el usuario Free intenta una tercera petición en el mismo segundo (14:23:16), el sistema la rechaza por exceder el límite, protegiendo así nuestros recursos. Una vez inicia un nuevo ciclo, el contador se reinicia.	In our Throttling demo, we set up two tenants: Premium with 5 requests per second and Free with 2. Running the simulation, we see both are accepted initially. However, when the Free user attempts a third request within the same second (14:23:16), the system rejects it for exceeding the limit, successfully protecting our resources. Once a new window starts, the counter resets.

```
=== THROTTLING PATTERN DEMONSTRATION ===
Two tenants with different limits:
- Premium: 5 requests/second
- Free: 2 requests/second

[14:23:15] Premium : ACCEPTED
[14:23:15] Premium : ACCEPTED
[14:23:16] Premium : ACCEPTED
[14:23:16] Free : ACCEPTED
[14:23:16] Free : ACCEPTED
[14:23:16] Premium : ACCEPTED
[14:23:16] Premium : ACCEPTED
[14:23:16] Free : REJECTED (limit exceeded)
--- Counter reset for new window ---
[14:23:17] Premium : ACCEPTED
```

Slide 16: Thanks

Español (Guion)	English (Script)
Eso concluye los ejemplos y la arquitectura detrás de estos patrones. Muchas gracias por su tiempo y atención. ¿Alguna pregunta?	That concludes the examples and the architecture behind these patterns. Thank you very much for your time and attention. Are there any questions?